AGENTS.md - Kaleido FlowTwin engineering contract

Kaleido FlowTwin

Álvaro Schwiedop Souto

Industrial AI & Predictive Quality Lead

EVOCON Solutions GmbH

15 julio 2026

Índice

1 AGENTS.md - Kaleido FlowTwin engineering contract

This file is normative for any human or agent implementing the MVP.

1.1 1. Mission

Build a read-only predictive operations layer for Kaleido that starts from sparse operational event data, produces immediate process intelligence, and promotes a sequential/world-model component only when it adds measured value.

Primary task:

> *Predict remaining time and material deviation risk for one Trace Port operation/shift, with calibrated uncertainty, useful lead time and an auditable explanation.*

The result must complement Trace Port, Shipping Board and Freight Intelligence. Do not build a competing generic logistics platform.

1.2 2. Read first

Before changing code, read completely:

1. `README.md`
2. `docs/investigacion/KALEIDO_RESEARCH.md`
3. `ARCHITECTURE.md`
4. `DATASETS.md`
5. `PLAN.md`
6. `docs/investigacion/SOTA_2026.md`
7. `DATA_REQUEST.md`

8. `docs/investigacion/REPO_REUSE_AUDIT.md`

Preserve the original evidence files:

- `correspondencia/CorreoEnviado1.txt`
- `correspondencia/CorreoRecibido1.txt`
- `correspondencia/adjuntos/Industrial_World_Model_MVP_es.pdf`

Never overwrite or reformat them.

1.3 3. Claim policy

Never claim `SOTA`, production-ready, causal, ROI, savings, accuracy, early warning or successful deployment without generated evidence.

Allowed states:

- `planned`: specified but not implemented;
- `smoke_only`: pipeline test, not business evidence;
- `diagnostic`: internal development result, test may be reused;
- `claim_eligible`: frozen protocol, clean commit, held-out test, provenance;
- `pilot_shadow`: running read-only with operator review;
- `validated_pilot`: pre-agreed acceptance gates passed.

Every result statement must name:

- dataset/export version and hash;
- split protocol;
- model and baselines;
- metric and uncertainty;
- number of seeds;
- threshold selection method;
- whether test influenced a choice;
- claim state.

Public/synthetic datasets prove pipeline competence only. Kaleido value requires Kaleido data.

1.4 4. Non-negotiable data rules

1.4.1 4.1 Actions are not context

An `action` is a timestamped decision that an operator/controller could change. Cargo type, vessel, customer, weather and shift identity are context. Static metadata must not support causal or counterfactual language.

Every adapter implements:

```
build_manifest()
validate_timestamps()
classify_fields()
build_object_graph()
build_grouped_splits()
run_leakage_audit()
```

Training fails closed when the audit fails. An explicit `--unsafe-debug` mode may create watermarked smoke artifacts that can never enter reports.

1.4.2 4.2 Plan revisions are immutable events

Never replace planned dates with latest values. Store each revision and its `valid_from`. A prediction at time `t` may only see the plan revision available at `t`.

1.4.3 4.3 Keep object identity

Project, operation, shift, cargo unit, resource, vessel, document and incident are separate object types. Do not flatten them into duplicated case rows without a reversible mapping.

1.4.4 4.4 Censoring and unknown outcomes

Incomplete operations are censored, not failures and not exact remaining time. Unknown incident status remains unknown. Use survival methods when appropriate.

1.4.5 4.5 No random row split

Default priority:

1. chronological future;
2. held-out project/client/port;
3. held-out cargo/operation type;
4. grouped cross-validation.

All prefixes/events from one operation remain in one partition.

1.5 5. Baselines before world models

Required floors:

- global and group median;
- persistence/rule thresholds;
- Cox/Kaplan-Meier or another censoring-aware model;
- regularized linear/logistic model;
- quantile gradient boosting/CatBoost;
- GRU/TCN;
- ProcessTransformer;
- object-centric graph baseline when relationships are available.

Do not remove a baseline because it wins.

Event-JEPA is permitted after M0/M1/M2 data and baseline gates. Required ablations:

- current prefix only;
- context only;
- correct actions;
- shuffled actions;
- action only;
- no JEPA objective;
- one horizon vs multi-horizon;
- no anticolapse regularizer;
- frozen vs fine-tuned encoder;
- object graph vs flattened trace.

If correct actions do not beat shuffled actions, do not claim action value.

1.6 6. Architecture boundaries

1.6.1 Ingestion

Read-only connectors, export parsing, source lineage and checksums. No model features beyond canonical normalization.

1.6.2 Semantic/event layer

Canonical IDs, timestamps, units, field roles, OCEL-like relationships and versioned plans. Unknown semantics stay unknown.

1.6.3 Analytics

Process discovery, conformance, variants, bottlenecks and descriptive reports. Must operate without ML training.

1.6.4 Prediction

Baselines, sequential models, Event-JEPA and uncertainty. Models never write source data.

1.6.5 Simulation/scenarios

Discrete-event simulation is separate from learned action ranking. A simulated saving is not a realized saving. Learned scenarios are restricted to observed support and approved actions.

1.6.6 Serving/UI

Versioned read-only API, dashboard, export and audit trail. Every prediction names its source event cutoff and plan revision.

1.7 7. Required repository structure

```
flowtwin/
  README.md
  AGENTS.md
  LICENSE
  SECURITY.md
  pyproject.toml
  uv.lock
  Makefile
  configs/
    data/
    model/
    experiment/
    deployment/
  src/flowtwin/
    cli.py
    config.py
    logging.py
    provenance.py
    data/
      contracts.py
      roles.py
      manifests.py
      timestamps.py
      object_graph.py
      splits.py
      leakage.py
      adapters/
        trace_port.py
        shipping_board.py
        freight_intelligence.py
        ocel.py
    process/
      discovery.py
      conformance.py
      variants.py
      bottlenecks.py
    features/
      prefix.py
      temporal.py
      categorical.py
      external_context.py
    baselines/
      naive.py
      survival.py
      boosting.py
```

```
      process_transformer.py
      object_graph.py
    models/
      contracts.py
      event_encoder.py
      target_encoder.py
      predictor.py
      event_jepa.py
      uncertainty.py
      heads.py
    simulation/
      discrete_event.py
      calibration.py
      scenarios.py
    evaluation/
      remaining_time.py
      risk.py
      calibration.py
      lead_time.py
      value.py
      sota_gate.py
    serving/
      api.py
      schemas.py
      model_registry.py
    dashboard/
      app.py
  scripts/
    audit_export.py
    build_ocel.py
    discover_process.py
    train_baselines.py
    train_event_jepa.py
    evaluate.py
    run_shadow_replay.py
    build_report.py
  tests/
    unit/
    integration/
    adversarial/
    fixtures/
  data/
    README.md
    manifests/
    schemas/
    splits/
  outputs/
  docs/
    progress.md
    decisions/
    data_cards/
    model_cards/
```

Do not commit raw Kaleido data, photos, credentials or customer identifiers.

1.8 8. Stack

- Python 3.11/3.12.
- `uv`, `ruff`, `mypy` or `pyright`, `pytest`.
- Polars or pandas; PyArrow/Parquet.

- Pydantic for contracts/config.
- PM4Py for XES/OCEL/process mining where license is accepted.
- scikit-learn and CatBoost/LightGBM as optional strong baselines.
- lifelines/scikit-survival as license/environment permits.
- PyTorch for sequential/JEPA models.
- FastAPI for read-only serving.
- Plotly/Altair or a small local web dashboard.
- SimPy for the first discrete-event simulator.

Dependencies are optional by capability; CPU smoke mode is mandatory. Record license and exact version.

1.9 9. Experiment contract

Every config includes:

```yaml
experiment_name:
hypothesis:
dataset_manifest:
schema_version:
split_manifest:
seed:
prediction_points:
horizons:
input_roles:
action_fields:
context_fields:
observation_fields:
outcome_fields:
forbidden_fields:
model:
baselines:
metrics:
threshold_selection:
calibration:
compute:
claim_state:
```

Each run writes:

```
run_manifest.json
config_resolved.yaml
environment.json
data_manifest.json
split_manifest.json
leakage_report.json
metrics.json
predictions.parquet
calibration.json
model_card.md
report.md
```

Record commit, dirty state, commands, start/end time and artifact hashes.

1.10 10. Metrics

Do not optimize only AUROC or next-event accuracy.

Remaining time:

- MAE, median AE, pinball loss;
- P50/P90 coverage and interval width;
- per operation/cargo/project and worst group.

Risk:

- AUPRC, event precision/recall;
- false alerts per shift/100 operations;
- lead time and missed events;
- Brier/ECE and reliability.

Process:

- conformance/deviation;
- waiting time and rework;
- variant coverage;
- bottleneck stability.

Business:

- time saved in reporting;
- avoidable wait/overtime under shadow simulation;
- operator acceptance/usefulness;
- estimated vs realized value clearly separated.

Thresholds are selected on validation only.

1.11 11. Adaptation

Never update the only reference model. Maintain:

- frozen reference;
- adaptive shadow/adapters;
- trust region;
- uncertainty/update gate;
- replay/anti-forgetting;
- signed/versioned artifact;
- rollback;
- explicit reset after process redesign.

Log every accepted/rejected update.

1.12 12. Security

- minimum-privilege read-only credentials;
- secrets from approved secret store only;
- input validation and size limits;
- pseudonymization mappings remain under Kaleido control;
- redact photos/notes in development fixtures;
- signed model/data artifacts for pilot;
- audit log for each prediction;
- no cross-customer training without explicit agreement;
- document GDPR roles and retention before ingest.

1.13 13. Tests

Unit:

- schema and role validation;
- timezone/DST and out-of-order timestamps;
- plan revision cutoff;
- OCEL relationship integrity;
- censoring;
- leakage;
- split disjointness;
- metrics/calibration;

- target encoder/adapter freeze.

Integration:

- synthetic OCEL -> audit -> process report;
- OCEL -> prefix dataset -> baseline -> evaluation;
- checkpoint save/load;
- CPU inference/API;
- offline shadow replay;
- report regeneration.

Adversarial:

- future outcome hidden in notes/columns;
- duplicate/retry events;
- timestamp rollback/DST;
- overwritten plan;
- shuffled actions;
- missing objects/modalities;
- new event vocabulary;
- drift and process redesign;
- customer/project ID shortcut;
- photo after prediction cutoff.

1.14 14. Milestone order

1. M0 bootstrap and synthetic fixture.
2. M1 canonical contract and audit CLI.
3. M2 OCEL/process-mining dashboard.
4. M3 naive/survival/boosting baselines.
5. M4 sequential and graph baselines.
6. M5 Event-JEPA only if gates allow.
7. M6 discrete-event scenarios.
8. M7 shadow replay/dashboard/API.
9. M8 pilot report and commercial evaluation.

Do not start M5 before M1-M4 produce valid evidence.

1.15 15. Definition of done

The MVP is done when:

- a fresh environment installs with one documented command;
- a Trace Port-like export is audited without manual code edits;
- process maps and data-quality report regenerate;
- baselines run on frozen grouped splits;
- prediction intervals and operational metrics are reported;
- Event-JEPA is either validated incrementally or honestly rejected;
- shadow replay exposes prediction cutoff, interval and reasons;
- no source write capability exists;
- security/data agreement checklist is complete;
- an operator review is documented;
- all tables are generated from artifacts;
- limitations and `what this does not prove` are explicit.

At the end of each task report: hypothesis, changes, tests, evidence, limitations and next falsifiable step.